

Fundamentos de la programación 2

Ejercicios. Hoja 1

Los ejercicios marcados como **Evaluable** tienen que subirse al CV durante la sesión de laboratorio.

- Se subirá **únicamente** el archivo **Program.cs**
- Sólo hará la entrega uno de los miembros del grupo.
- La línea 1 y siguientes deben contener los nombres de los alumnos que hacen la entrega (uno por línea) de la forma:

```
// Nombre Apellido1 Apellido2
// Nombre Apellido1 Apellido2
```

Consideremos la representación para el tipo de los polinomios vista en clase:

```
const int N = 100; // tamaño de los arrays de monomios
struct Monomio {    // coeficiente y exponente
    public double coef;
    public int exp;
}

struct Polinomio {
    public Monomio [] mon; // array de monomios
    public int nMons;      // num de monomios = primera pos libre en el array mon
}
```

Y consideremos los *invariantes de representación*¹ (también vistos en clase):

- Para todos los monomios del polinomio se verifica $\text{exp} \geq 0$ y $\text{coef} \neq 0$.
- No existen dos monomios con el mismo exponente en el array de monomios ($\text{mon}[i].\text{exp} \neq \text{mon}[j].\text{exp} \forall i, j$ con $0 \leq i < j < \text{nMon}$).

1. (**Evaluable**) Implementar el método:

- **Polinomio Derivada(Polinomio p)**: devuelve el polinomio resultante de derivar el polinomio p.

2. Consideremos la representación anterior con los invariantes mencionados y uno adicional:

- Los monomios del array están ordenados en orden creciente de acuerdo a sus exponentes (de menor a mayor exponente).

A continuación implementaremos las siguientes operaciones:

- **void Inserta(Monomio m, Polinomio p)**: añade el monomio m al polinomio p. Producirá error cuando la estructura (el array de monomios) esté llena y se requiera más espacio.

Para buscar el lugar del monomio m en el array de monomios p.mon puede hacerse una búsqueda lineal (no es necesario implementar una búsqueda más elaborada).

Deben implementarse dos métodos auxiliares:

¹El término *invariantes de la representación* quiere decir que todos los métodos que reciben un polinomio deben asumir que se verifica esta pre-condición y deben preservarla como post-condición en los polinomios que devuelvan.

- `void DesplazaDer(Polinomio p, int pos)`: desplaza una posición a la derecha todos los monomios de `p` del intervalo `[pos,nMons-1]` (abre un hueco en `pos` para insertar ahí un nuevo monomio). Si no hay hueco escribirá un mensaje de error.
- `void DesplazaIzq(Polinomio p, int pos)`: desplaza una posición a la izquierda todos los monomios de `p` del intervalo `[pos,p.nMons-1]` (elimina el monomio del posición `pos`).
- `void Lee(Polinomio p)`: lee un polinomio de teclado (se puede utilizar la versión vista en clase o mejorarla).
- `void Escribe(Polinomio p)`: escribe en pantalla un polinomio (mejorar la versión vista en clase evitando operadores innecesarios).
- `int Grado(Polinomio p)`: devuelve el grado del polinomio `p`.
- `Polinomio Suma(Polinomio p1, Polinomio p2)`: devuelve el polinomio resultante de sumar `p1` y `p2`. Producirá error si el polinomio resultante no cabe en la estructura. Implementar también la **Resta**.
- `double Evalua(Polinomio p,double v)`: evalúa el polinomio `p` en el valor `v`.
- `Polinomio Multiplica(Polinomio p1, Polinomio p2)`: calcula la multiplicación de `p1` y `p2`. Producirá error si el polinomio resultante no cabe en la estructura.
- `void Divide(Polinomio p1,Polinomio p2,Polinomio c,Polinomio r)`: calcula el cociente `c` y el resto `r` resultantes de la división de `p1` entre `p2`.
- `bool Iguales(Polinomio p1, Polinomio p2)`: determina si son iguales los polinomios representados en `p1` y `p2`.

Hay otras operaciones interesantes que pueden implementarse:

- `Polinomio Convierte(Polinomio p)`: dado un polinomio `p` sin restricciones sobre los monomios que contiene lo convierte en un polinomio equivalente con todos los invariantes mencionados arriba, incluido el orden.
- `void Salva(Polinomio p)`: solicita un nombre de archivo y guarda en el mismo el contenido del polinomio `p` en un formato razonable y fácilmente legible.
- `void Restaura(Polinomio p)`: solicita un nombre de archivo y lee en `p` los datos de un polinomio guardado en el mismo formato que el método anterior.
- `void Extiende(Polinomio p)`: duplica el tamaño del array de monomios de `p` almacenando los monomios existentes (para conseguir esto, en realidad hay que crear un nuevo vector de monomios y copiar el antiguo). Esta operación operará de *modo silencioso* y será útil para ampliar el array de monomios cuando se necesite. Las operaciones pedidas arriba ya no darán errores de desbordamiento la estructura.